QUEUE SIMULATION

A PROJECT REPORT

Submitted by

SANJEEV KUMAR R

(2303811720521044)

in partial fulfillment of requirements for the award of the course

CGB1122 – DATA STRUCTURES

in

INFORMATION TECHNOLOGY

K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(An Autonomous Institution, affiliated to Anna University Chennai and Approved by AICTE, New Delhi)

SAMAYAPURAM – 621 112

May, 2024

K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(AUTONOMOUS)

SAMAYAPURAM – 621 112

BONAFIDE CERTIFICATE

Certified that this project report titled "**Queue Simulation" for Call Center with Customer Service Using Available Agents** is the bonafide work of **SANJEEV KUMAR R (2303811720521044),** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported here in does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Mrs.A.Delphin Carolina Rani, M.E.,Ph.D.,

HEAD OF THE DEPARTMENT

PROFESSOR

Department of IT

K. Ramakrishnan College of Technology (Autonomous)

Samayapuram–621112.

SIGNATURE

Mr.D.P.Devan B.E., M.E.,

SUPERVISOR

ASSISTANT PROFESSOR

Department of CSE

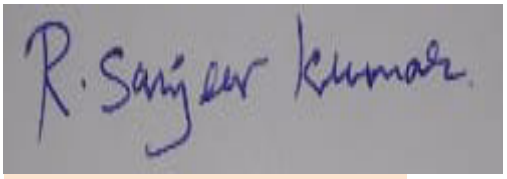
K. Ramakrishnan College of Technology (Autonomous)

Samayapuram–621112.

Submitted for the viva-voce examination held on 16/06/2024

DECLARATION

I declare that the project report on "**Queue Simulation" for Call Center with Customer Service Using Available Agents** is the result of original work done by us and best of our knowledge, similar work has not been submitted to **"ANNA UNIVERSITY CHENNAI"** for the requirement of Degree of **BACHELOR OF TECHNOLOGY**. This project report is submitted on the partial fulfillment of the requirement of the award of the course **CGB1122- DATA STRUCTURES.**



Signature

_____

SANJEEV KUMAR R

Place: Samayapuram

Date: 16/06/2024

Internal sign

ACKNOWLEDGEMENT

It is with great pride that I express our gratitude and indebtedness to our institution, "**K. Ramakrishnan College of Technology (Autonomous)**", for providing us with the opportunity to do this project.

I extend our sincere acknowledgment and appreciation to the esteemed and honorable Chairman, **Dr. K. RAMAKRISHNAN**, **B.E.,** for having provided the facilities during the course of our study in college.

I would like to express our sincere thanks to our beloved Executive Director, **Dr. S. KUPPUSAMY, MBA, Ph.D.,** for forwarding our project and offering an adequate duration to complete it.

I would like to thank **Dr. N. VASUDEVAN, M.TECH., Ph.D.,**

Principal, who gave the opportunity to frame the project to full satisfaction.

I thank **Dr. A. DELPHIN CAROLINA RANI M.E., Ph.D.,** Head of the Department of INFORMATION TECHNOLOGY , for providing her encouragement in pursuing this project.

I wish to convey our profound and heartfelt gratitude to our esteemed project guide **D.P.DEVAN B.E., M.E.,** Department of **COMPUTER SCIENCE AND ENGINEERING,** for her incalculable suggestions, creativity, assistance and patience, which motivated us to carry out this project.

I render our sincere thanks to the Course Coordinator and other staff members for providing valuable information during the course.

I wish to express our special thanks to the officials and Lab Technicians of our departments who rendered their help during the period of the work progress.

VISION OF THE INSTITUTION

To emerge as a leader among the top institutions in the field of technical education.

MISSION OF THE INSTITUTION

- Produce smart technocrats with empirical knowledge who can surmount the global challenges.
- Create a diverse, fully-engaged, learner-centric campus environment to provide quality education to the students.
- Maintain mutually beneficial partnerships with our alumni, industry, and Professional associations.

VISION OF DEPARTMENT

To be a center of eminence in creating competent software professionals with research and innovative skills.

MISSION OF DEPARTMENT

M1: Industry Specific: To nurture students in working with various hardware and software platforms inclined with the best practices of industry.

M2: Research: To prepare students for research-oriented activities.

M3: Society: To empower students with the required skills to solve complex technological problems ofsociety.

PROGRAM EDUCATIONAL OBJECTIVES

1. PEO1: Domain Knowledge

To produce graduates who have strong foundation of knowledge and skills in the field of Computer Science and Engineering.

2. PEO2: Employability Skills and Research

To produce graduates who are employable in industries/public sector/research organizations or work as an entrepreneur.

3. PEO3: Ethics and Values

To develop leadership skills and ethically collaborate with society to tackle real-world challenges.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO 1: Domain Knowledge

To analyze, design and develop computing solutions by applying foundational concepts of Computer Science and Engineering.

PSO 2: Quality Software

To apply software engineering principles and practices for developing quality software for scientific and business applications.

PSO 3: Innovation Ideas

To adapt to emerging Information and Communication Technologies (ICT) to innovate ideas and solutions to existing/novel problems

PROGRAM OUTCOMES (POs)

Engineering students will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities

with an understanding of the limitations

6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

ABSTRACT

This project presents a queue simulation system designed for call centers, where customers are serviced by available agents. The system utilizes a priority queue data structure to manage customer arrivals, ensuring that higher-priority customers are serviced first. Agents are dynamically allocated to handle incoming calls based on their availability. The architecture includes modules for customer arrival, queue management, agent pool, service process, and metrics reporting. The simulation provides an interactive menu for adding customers and agents, viewing the queue, monitoring service processes, and displaying updated performance metrics. This simulation aims to optimize call center operations by effectively managing resources and improving customer satisfaction.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGENO.
	ABSTRACT	viii
	LIST OF FIGURES	x
	LIST OF ABBREVIATIONS	xi
1	**INTRODUCTION**	
	1.1 Introduction	1
	1.2 Purpose And Importance	1
	1.3 Objectives	2
	1.4 Project Summarization	2
2	**PROJECT METHODOLOGY**	
	2.1. Introduction to System Architecture	3
	2.2 Detailed System Architecture Diagram	4
3	**DATA STRUCTURE PREFERANCE**	
	3.1 Explanation of why a Queue Data structure was chosen	5
	3.2 Comparison with Other Data Structures	6
	3.3 Advantages and disadvantages of using Queue Data structure	6
4	**DATA STRUCTURE METHODOLOGY**	
	4.1. Priority Queue	7
	4.2. Hash Table	7
		8
5	**MODULES**	
	5.1 Customer Arrival	9
		9
		9
	5.2 Service	
	5.3 Metrics And Reporting	
6	**CONCLUSION & FUTURE SCOPE**	
	6.1 Conclusion	10
	6.2 Future Scope	10
	APPENDICES	
	Appendix A-Source code	12
	Appendix B -Screen shots	18

LIST OF FIGURES

FIGURE NO	TITLE	PAGENO.
2.1	Architecture Diagram	4

LIST OF ABBREVIATIONS

ABBREVIATIONS

DLL	-	Doubly Linked List
UI	-	User Interface
CLI	-	Command Line Interface
GUI	-	Graphical User Interface
NLP	-	Natural Language Processing
GUI	-	Graphical User Interface
APIs	-	Application Programming Interface

CHAPTER 1
INTRODUCTION
1.1 INTRODUCTION TO PROJECT

In the fast-paced environment of a call center, managing customer wait times and ensuring efficient service delivery are critical for maintaining high levels of customer satisfaction and operational efficiency. Queue simulation is a powerful tool that models the dynamic interactions between incoming customer calls, queue management, and agent availability. This simulation allows for the analysis and optimization of call center operations by replicating real-world scenarios, including fluctuating call volumes and variable service times. By examining various performance metrics such as average wait times, service levels, and agent utilization rates, call centers can identify bottlenecks and implement strategies to improve their service processes. This study focuses on developing a discrete-event simulation model to evaluate the impact of different operational strategies on the overall performance of a call center.

1.2 PURPOSE AND IMPORTANCE OF THE PROJECT

The purpose of queue simulation in a call center is to create a realistic model that helps in understanding and optimizing the flow of customer interactions, from arrival to service completion. By simulating the arrival of customer calls, the waiting times, and the service processes managed by available agents, the model aims to identify inefficiencies and test various strategies for improvement.

The importance of queue simulation lies in its ability to provide actionable insights without disrupting actual operations. It helps call center managers to:

1. Optimize Resource Allocation Determine the optimal number of agents needed to handle varying call volumes while minimizing costs.

2. Improve Customer Satisfaction Reduce average wait times and ensure timely service, thereby enhancing the overall customer experience.

3. Enhance Operational Efficience Identify and eliminate bottlenecks in the

service process, leading to smoother and more efficient operations.

4. Test Scenarios Safely evaluate the impact of different staffing levels, call routing strategies, and other operational changes before implementing them in real-life situations.

Overall, queue simulation is a vital tool for strategic planning and operational management in call centers, leading to improved performance and customer satisfaction.

1.3 OBJECTIVES

1. Minimize Customer Wait Time
2. Optimize Agent Utilization
3. Improve Service Levels
4. Analyze System Performance
5. Enhance Customer Satisfaction

1.4 PROJECT SUMMARIZATION

Queue simulation in a call center models customer arrivals, waiting times, and service processes to optimize efficiency and service quality. By analyzing key metrics such as wait times, agent utilization, and service levels, the simulation identifies inefficiencies and tests different strategies. This approach helps improve resource allocation, enhance customer satisfaction, and support data-driven decision-making, leading to a more effective and responsive call center operation.

- Scalability Testing: Allows the call center to test how well it can handle varying volumes of incoming calls and scale resources accordingly.
- Predictive Analysis: Utilizes historical data to forecast future trends and prepare for seasonal fluctuations in call volumes.
- Staff Training: Identifies peak times and optimal staffing levels, aiding

in better scheduling and training of new agents to handle high call volumes efficiently.

- Cost Efficiency: Helps balance operational costs with service quality by optimizing the number of active agents based on real-time demand.
- Customer Retention: By improving response times and service quality, the simulation aids in retaining customers and enhancing brand loyalty.

CHAPTER 2

PROJECT METHODOLOGY

2.1 INTRODUCTION TO SYSTEM ARCHITECTURE

The system architecture for queue simulation in a call center is designed to model the flow of customer interactions, from initial contact to service completion. This architecture encompasses several interconnected modules that collectively simulate the arrival of customer calls, their waiting times in the queue, and the servicing process by available agents. Key components include the Customer Arrival Module, Queue Module, Agent Pool Module, Service Module, Departure Module, and Metrics and Reporting Module.

2.1.1 High-Level System Architecture

The high-level system architecture for the phone directory application typically consists of several key components:

(i) Customer Arrival Module
(ii) Service Module
(iii) Metrics and Reporting Module

2.1.2 Components of the System Architecture

1. Customer Arrival Module Function: Simulates the arrival of customer calls based on predefined patterns (e.g., Poisson distribution).Inputs: Arrival rate parameters.Outputs: Customer entities entering the system.

2. Service Module Function: Simulates the interaction between agents and customers, including the service process. Inputs: Customer entities from the Queue Module, available agents from the Agent Pool Module.Outputs: Serviced customer entities, updated agent status (busy or available).

5.Departure Module: Records details of serviced customers in a list or log file. This module ensures that service completions are logged for reporting and performance analysis.

3.Metrics and Reporting module: Collects and stores performance data in

aggregated structures or databases. Computes metrics and generates reports to analyze call center efficiency and identify areas for improvement.

2.2 DETAILED SYSTEM ARCHITECTURE DIAGRAM

This architecture illustrates the detailed interactions and functionalities of each module involved in simulating and managing the queue in a call center environment.

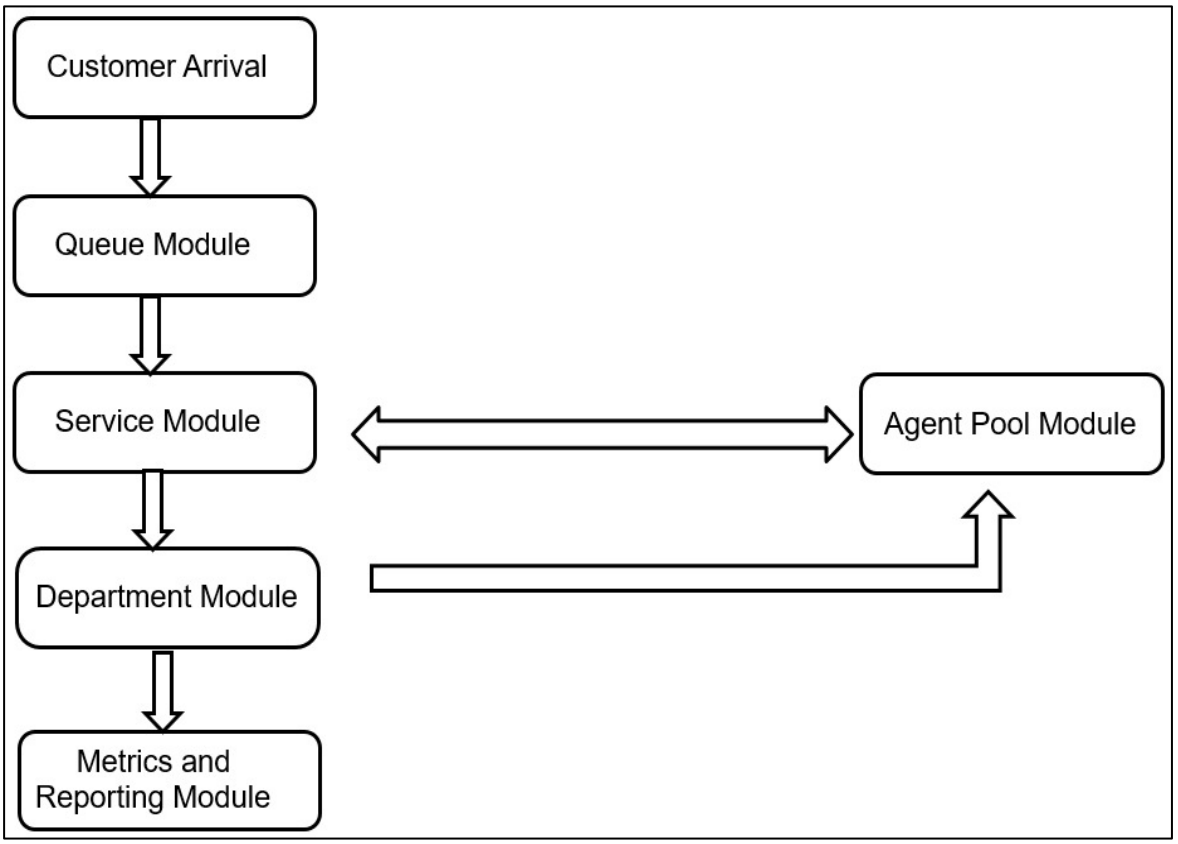


Fig 2.1 : Architecture Diagram

CHAPTER 3

DATA STRUCTURE PREFERANCE

3.1 EXPLANATION OF WHY A QUEUE DATA STRUCTURE WAS CHOSEN

For a queue simulation in a call center where customers are serviced using available agents, the preferred data structures depend on the specific requirements of the simulation and the programming language being used. However, here are some commonly used data structures that are suitable for managing the queue and agent pool:

1. Queue Data Structure

Array-based queue: Provides constant-time enqueue and dequeue operations, but may require resizing if the capacity is exceeded.

Linked list-based queue: Dynamic in size and doesn't require resizing, but may have slightly higher overhead due to memory allocation.

Priority queue: Allows for prioritizing customers based on certain criteria (e.g., urgency of the issue), ensuring that high-priority customers are serviced first.

2. Agent Pool Data Structure

Array or list-based structure: Stores information about available agents, such as their status (busy or available) and any relevant attributes (e.g., skills, expertise).

Hash table: Provides fast access to agents based on unique identifiers (e.g., agent ID), allowing for efficient lookup and modification of agent information.

3. Customer and Agent Objects

Object-oriented approach: Define classes for customers and agents, encapsulating relevant attributes and behaviors (e.g., customer ID, agent availability status).

Structured data format: Use dictionaries or JSON objects to represent customers and agents, with key-value pairs for storing attributes and state information.

4. Performance Considerations

Consider the performance characteristics of the data structures, such as time complexity for insertion, deletion, and lookup operations, as well as memory usage.

Choose data structures that strike a balance between efficiency and ease of implementation, considering factors such as the expected size of the queue and agent pool, and the frequency of operations performed on them.

Overall, the choice of data structures should prioritize simplicity, efficiency, and suitability for the specific requirements of the queue simulation in the call center environment.

3.2 COMPARISON WITH OTHER DATA STRUCTURES

In queue simulation for call centers, array-based queues offer simplicity and efficient memory usage with constant-time enqueue and dequeue operations. However, they require resizing if capacity is exceeded, potentially impacting performance. Linked list-based queues, while dynamic and avoiding resizing, may incur slightly higher overhead due to memory allocation. Priority queues prioritize customers based on criteria such as urgency, ensuring high-priority customers are serviced first, but implementation complexity and performance overhead can be higher. Hash table-based agent pools provide fast access to agent information but may have higher memory overhead and require collision handling.

3.3 ADVANTAGES AND DISADVANTAGES OF USING QUEUE DATA STRUCTURE

3.3.1 Advantages of Using a Queue Data structure

The queue data structure offers several advantages in queue simulation for call centers where customers are serviced using available agents. Firstly, it facilitates orderly customer management, ensuring that customers are serviced in the order they arrive, maintaining fairness and transparency in service delivery. Secondly, queues support efficient customer waiting management, allowing customers to wait in a structured manner until an agent becomes available, minimizing confusion and

frustration. Thirdly, the FIFO (First-In-First-Out) nature of queues ensures that customers are serviced in a predictable and systematic manner, contributing to the overall efficiency of call center operations. Lastly, queues enable easy implementation and integration within the simulation framework, providing a straightforward and effective solution for managing customer flow in the call center environment.

3.3.2 Disadvantages of Using a Queue Data structure

One disadvantage of using queue data structure in queue simulation for call centers is its static nature, which may lead to inefficiencies in handling fluctuating call volumes. Additionally, queues may suffer from performance degradation when large numbers of customers accumulate, as resizing operations can be time-consuming. Furthermore, queues may not adequately prioritize customers based on urgency or other criteria, potentially resulting in delays for high-priority cases. Lastly, if not managed properly, queues may introduce complexity in handling exceptions or special cases, requiring additional logic to ensure fair and efficient service delivery.

CHAPTER -4

DATA STRUCTURE METHODOLOGY

4.1 Priority Queue

A priority queue is a data structure that manages elements based on their priority, ensuring that the highest-priority elements are served before others. This is particularly useful in a call center where some customers may need to be prioritized over others based on urgency or other criteria.

4.1.1 Key features of a Priority Queue

1. Priority-based Service: Customers are serviced based on urgency, ensuring high-priority cases are addressed promptly, improving customer satisfaction.
2. Efficient Management: Provides O(log n) time complexity for inserting and removing customers, maintaining performance even with high volumes.
3. Dynamic Adjustments: Allows real-time priority changes for customers, adapting to shifting service needs.
4. Fair Allocation: Ensures customers are serviced in priority order, enhancing fairness and transparency.
5. Scalability: Handles increasing numbers of customers without significant performance loss, supporting call center growth.

CHAPTER-5

MODULES

5.1 CUSTOMER ARRIVAL

Description: The Customer Arrival Module in the queue simulation for a call center is responsible for managing incoming customer data. In this module, new customer arrivals are recorded and stored in a dynamic list or array for processing. Each customer is represented by a structure containing relevant information such as customer ID and arrival time.

5.2. AGENT

Agents are the call center representatives responsible for handling customer calls. Each agent is identified by a unique ID and has an availability status indicating whether they are free to take a call or currently busy. The system dynamically allocates available agents to incoming customer calls, optimizing resource utilization. Agents' status is updated in real-time to reflect their engagement in the service process.

5.2 SERVICE

Description: The Service Module manages the ongoing interactions between agents and customers in the call center. In this module, a simple queue is typically used to track which agents are currently engaged in service and when they become available again.

5.3 METRICS REPORTING

Description: The Metrics and Reporting Module collects and stores performance data for analysis and reporting. In this module, performance metrics such as total customers served, average wait time, and service duration are computed and stored.

CHAPTER 6
CONCLUSION & FUTURE SCOPE

6.1 CONCLUSION

In a call center where customers are serviced using available agents, effective queue simulation is essential to ensure optimal operational efficiency and customer satisfaction. Implementing priority queues allows for the prioritization of customer service based on urgency, ensuring that high-priority customers receive timely attention. Concurrently, using hash tables to manage agent availability provides fast and efficient updates and lookups, facilitating dynamic and scalable workforce management. Together, these data structures enable a robust, scalable, and flexible system that can adapt to varying workloads and service requirements. Ultimately, leveraging these methodologies helps maintain high service levels, reduces customer wait times, and maximizes agent utilization, contributing to the overall success of the call center.

6.2 FUTURE SCOPE

The future scope for queue simulation in call centers is vast and promising, driven by advancements in technology and evolving customer expectations. Integration of artificial intelligence and machine learning can revolutionize call centers through predictive analytics, smart routing, and dynamic priority adjustments, enhancing efficiency and personalization. Real-time performance monitoring with advanced dashboards and instant alerts will enable proactive management of call volumes and service levels. Extending queue management to an omni-channel approach will ensure a consistent customer experience across all communication platforms. Cloud-based solutions will offer scalability and global accessibility, supporting distributed operations. Enhanced agent management, including skill-based routing and automated workforce management, will optimize resource allocation and improve service quality. These advancements will collectively transform call centers into highly efficient, responsive, and customer-centric operations.

APPENDICES

APPENDIX A-SOURCE CODE

```
  #include <stdio.h>
#include <stdlib.h>

#define MAX_CUSTOMERS 100
#define MAX_AGENTS 5

typedef struct {
    int customerID;
} Customer;

typedef struct {
    Customer customers[MAX_CUSTOMERS];
    int front, rear, size;
} Queue;

void initQueue(Queue *q) {
    q->front = 0;
    q->rear = -1;
    q->size = 0;
}

int isFull(Queue *q) {
    return q->size == MAX_CUSTOMERS;
}

int isEmpty(Queue *q) {
    return q->size == 0;
```

```
}

void enqueue(Queue *q, int customerID) {
    if (isFull(q)) {
        printf("Queue is full. Cannot add more customers.\n");
        return;
    }
    q->rear = (q->rear + 1) % MAX_CUSTOMERS;
    q->customers[q->rear].customerID = customerID;
    q->size++;
}

int dequeue(Queue *q) {
    if (isEmpty(q)) {
        printf("Queue is empty. No customers to serve.\n");
        return -1;
    }
    int customerID = q->customers[q->front].customerID;
    q->front = (q->front + 1) % MAX_CUSTOMERS;
    q->size--;
    return customerID;
}

int main() {
    Queue customerQueue;
    initQueue(&customerQueue);

    int agents[MAX_AGENTS] = {0}; // 0 means agent is free, 1 means agent is
    busy

    int customerID = 1;
```

```
char command;

while (1) {
    printf("Enter 'a' to add customer, 's' to serve customer, 'q' to quit: ");
    scanf(" %c", &command);

    if (command == 'a') {
        enqueue(&customerQueue, customerID++);
    } else if (command == 's') {
        int served = 0;
        for (int i = 0; i < MAX_AGENTS; i++) {
            if (agents[i] == 0) {
                int customer = dequeue(&customerQueue);
                if (customer != -1) {
                    agents[i] = 1;
                    printf("Agent %d is serving Customer %d\n", i + 1, customer);
                    served = 1;
                    // Simulate serving the customer (you can add sleep or delay here)
                    agents[i] = 0; // Mark agent as free after serving
                }
                break;
            }
        }
        if (!served) {
            printf("All agents are busy. Please wait.\n");
        }
    } else if (command == 'q') {
        break;
    } else {
        printf("Invalid command.\n");
    }
```

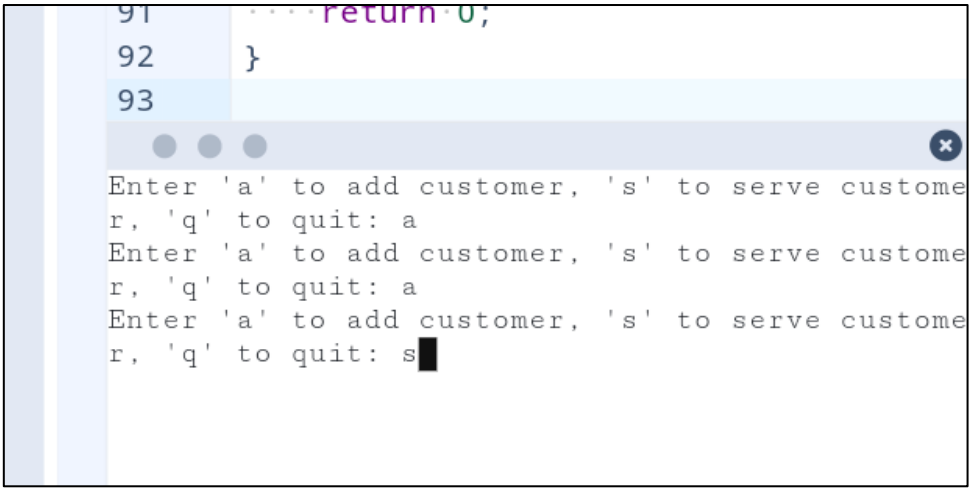
```
    }

    return 0;
}
```

APPENDIX B - SCREENSHOTS

RESULT AND DISCUSSION

Add Customer



Agent

```
66        } else if (command == 's') {
67            int served = 0;
```

```
Enter 'a' to add customer, 's' to serve custome
r, 'q' to quit: s
Queue is empty. No customers to serve.
All agents are busy. Please wait.
Enter 'a' to add customer, 's' to serve custome
r, 'q' to quit:
```

Enter input:

Service Process

```
93

Enter 'a' to add customer, 's' to serve custome
r, 'q' to quit: a
Enter 'a' to add customer, 's' to serve custome
r, 'q' to quit: a
Enter 'a' to add customer, 's' to serve custome
r, 'q' to quit: s
Agent 1 is serving Customer 1
Enter 'a' to add customer, 's' to serve custome
r, 'q' to quit: s
Agent 1 is serving Customer 2
Enter input:
E o add customer, 's' to serve custome
r, 'q' to quit:
```

Metrics and Reporting

```
    return 0;
}

Enter 'a' to add customer, 's' to serve custome
r, 'q' to quit: a
Enter 'a' to add customer, 's' to serve custome
r, 'q' to quit: a
Enter 'a' to add customer, 's' to serve custome
r, 'q' to quit: s
Agent 1 is serving Customer 1
Enter 'a' to add customer, 's' to serve custome
r, 'q' to quit: s
Agent 1 is serving Customer 2
Enter 'a' to add customer, 's' to serve custome
r, 'q' to quit: q
=== YOUR PROGRAM HAS ENDED ===
```